

API

NestJS 11 over Prisma 6 and PostgreSQL 16. Every route is prefixed `/api`, so the base URL in development is `http://localhost:4000/api`.

An interactive reference generated from the code is at <http://localhost:4000/api/docs> (Swagger). This document covers the conventions the endpoints share and lists them in one place.

Conventions

Authentication

Every route is authenticated by default. A global guard validates the bearer token and loads the user; a route opts out with `@Public()`, which only `POST /auth/login` and `GET /api/health` do.

```
Authorization: Bearer <jwt>
```

The token is re-checked against the database on every request, so a deactivated account or a changed role takes effect immediately rather than at expiry.

Authorisation

Controllers declare `@RequirePermission(module, action)`, checked against the matrix in `shared/src/rbac.ts`. Failing it is a `403`.

Own-records scoping

The `EMPLOYEE` role only ever sees rows tied to its own employee record. This is applied in the query, not the matrix, and it is not something the client can influence: an `employeeId` in the URL or body is replaced with the caller's own. An account with no linked employee resolves to an id nothing matches, so it sees an empty list rather than everyone's.

Ids

Every primary key is a `UUIDv7` — validated by `IsEntityId` on DTOs and `ParseEntityIdPipe` on route parameters, not by `IsUUID` / `ParseUUIDPipe`. A malformed id is a `400`, never a database error. See [architecture.md](#) for why the looser validator is the right one.

Request bodies

Validated with `class-validator`. Unknown fields are **rejected**, not stripped: sending a property the DTO does not declare returns `400`, which catches a client sending a derived value the API computes for itself.

Errors come back as:

```
{ "statusCode": 400, "message": ["wage must not be less than 0"], "error": "Bad Request" }
```

Each message leads with the property name, which is what lets the web client put it under the right field.

Deletes

Several deletes **archive rather than remove** once payroll history refers to the record — an employee with payslips is marked inactive with an exit date; a contract referenced by a payslip is cancelled. The response says which happened:

```
{ "deleted": false, "archived": true }
```

Report the outcome from that flag rather than assuming a delete.

Money and dates

Currency and hour columns are **NUMERIC** in the database, normalised to plain JSON numbers at the boundary. Dates cross as ISO strings.

Endpoints

Auth

Method	Path	Permission
POST	/auth/login	public
GET	/auth/me	signed in
POST	/auth/change-password	signed in

login returns `{ accessToken, user }`. **me** returns the current **AuthUser**, and is how the web client re-checks a role that an admin may have changed.

Both carry **mustChangePassword**. It is true while the password in use is one the system issued, and every guarded page redirects to **/change-password** until it is false — an issued password gets someone in and no further.

change-password takes `{ currentPassword, newPassword }`. It verifies the current password, refuses a new one identical to it, and clears the flag. The confirmation field is a browser concern and is not sent.

Health

Method	Path	Permission
GET	/health	public

Returns `{ status, database, timestamp }`. Compose uses it to hold the web container back until the API has finished migrating.

Employees

Method	Path	Permission
GET	/employees	employees:read
GET	/employees/:id	employees:read
POST	/employees	employees:create
PATCH	/employees/:id	employees:update
DELETE	/employees/:id	employees:delete
POST	/employees/:id/reinvite	employees:update

Query: `q`, `departmentId`, `employeeType`, `status`, `missingBank`. The list returns `EmployeeSummaryDto`; the single record returns `EmployeeDetailDto`, which adds the personal, bank and relation-id fields plus related-record counts.

`POST /employees` also creates the sign-in account, in one transaction with the employee — there is no separate step and no way to end up with one without the other. It generates a one-time password, mails it, and records the attempt in the outbox with the password masked. `canSignIn: false` creates the account inactive and sends nothing.

The create response carries `invite`. When the mail could not be delivered — no transport configured, or the send failed — it reports `delivered: false` with the reason **and the one-time password**, because that is the only copy: the account holds a hash and the outbox holds a masked body. Whoever created the employee is the one person who can pass it on, and the web client shows it once in a dialog that must be dismissed. `invitedAt` is set only on a real delivery.

`reinvite` issues a fresh one-time password, mails it and reactivates the account. It is the way back for anyone whose account has no usable password — including the accounts the identity migration created for employees that predated it. It returns `{ delivered, error?, oneTimePassword? }` rather than throwing, on the same terms as create: a send that fails still leaves a usable account, a row saying why, and the password in the response.

`DELETE` removes the account with the employee. Where payslips exist, both are deactivated instead, so payroll history keeps the person it belongs to.

Contracts

Method	Path	Permission
GET	/contracts	contracts:read
GET	/contracts/:id	contracts:read

Method	Path	Permission
POST	/contracts	contracts:create
PATCH	/contracts/:id	contracts:update
DELETE	/contracts/:id	contracts:delete

Query: `q`, `employeeId`, `status`, `expiring`, `periodStart`, `periodEnd`. Passing a period sets `isApplicableForPeriod` on each row — the flag payroll uses to pick the governing contract.

Creating a **RUNNING** contract that overlaps another is rejected. `employeeId` is ignored on update: a contract cannot be moved to a different person.

Attendance

Method	Path	Permission
GET	/attendance	attendance:read
GET	/attendance/summary	attendance:read
POST	/attendance/check-in	attendance:create
POST	/attendance/check-out	attendance:create
GET	/attendance/:id	attendance:read
POST	/attendance	attendance:create
PATCH	/attendance/:id	attendance:update
DELETE	/attendance/:id	attendance:delete

Query: `q`, `employeeId`, `status`, `from`, `to`, `limit`.

Worked hours, overtime and status are **derived** from the punches and that weekday's schedule. A **PATCH** carrying `status` overrides the derived value and flags the record as manually edited, recording who, when and why. `check-in` and `check-out` always act on the caller's own record.

Time off

Method	Path	Permission
GET	/time-off/types	timeOffTypes:read
POST	/time-off/types	timeOffTypes:create
PATCH	/time-off/types/:id	timeOffTypes:update
DELETE	/time-off/types/:id	timeOffTypes:delete
GET	/time-off/balances/:employeeId	timeOffAllocations:read
GET	/time-off/allocations	timeOffAllocations:read

Method	Path	Permission
GET	/time-off/allocations/:id	timeOffAllocations:read
POST	/time-off/allocations	timeOffAllocations:create
PATCH	/time-off/allocations/:id	timeOffAllocations:update
POST	/time-off/allocations/:id/approve	timeOffAllocations:approve
POST	/time-off/allocations/:id/refuse	timeOffAllocations:approve
DELETE	/time-off/allocations/:id	timeOffAllocations:delete
GET	/time-off/requests	timeOffRequests:read
GET	/time-off/requests/:id	timeOffRequests:read
POST	/time-off/requests	timeOffRequests:create
PATCH	/time-off/requests/:id	timeOffRequests:update
POST	/time-off/requests/:id/approve	timeOffRequests:approve
POST	/time-off/requests/:id/refuse	timeOffRequests:approve
POST	/time-off/requests/:id/cancel	timeOffRequests:read
DELETE	/time-off/requests/:id	timeOffRequests:delete

balances is derived, never stored: approved allocations minus approved requests, per type. Approving a request links it to a specific allocation, so consumption is auditable. A request is refused when it overlaps existing leave, exceeds the remaining balance, or breaches the type's per-request cap. Duration counts only scheduled working days.

refuse accepts an optional `{ reason }`.

Payroll

Method	Path	Permission
GET	/payruns	payruns:read
GET	/payruns/eligible-employees	payruns:create
GET	/payruns/:id	payruns:read
POST	/payruns	payruns:create
POST	/payruns/:id/compute	payruns:update
POST	/payruns/:id/validate	payruns:update
POST	/payruns/:id/mark-paid	payruns:update
POST	/payruns/:id/send-payslips	payruns:update
DELETE	/payruns/:id	payruns:delete

Method	Path	Permission
GET	/payslips	payslips:read
GET	/payslips/:id	payslips:read
POST	/payslips/:id/recompute	payslips:update
GET	/payslips/:id/pdf	signed in
GET	/email-logs	payslips:read

`eligible-employees` takes `periodStart`, `periodEnd`, `structureId` and optionally `departmentId` and `employeeType`, and returns everyone in scope with an `eligible` flag, a `reason` when they are not, and a `warning` when they are but something is off. It is a read: it creates nothing.

The lifecycle is `DRAFT` → `COMPUTED` → `VALIDATED` → `PAID`. Validation is blocked while any warning stands.

`send-payslips` generates each PDF, attaches it, and records every attempt in `EmailLog` with its status and, on failure, the provider's own message. Delivery goes through Azure Communication Services when it is configured, SMTP when only that is, and otherwise nowhere — the attempt is still recorded, so the flow is demonstrable without credentials. It returns `{ sent, failed }`.

The PDF is a binary stream. A browser cannot set an `Authorization` header on a plain link, so the web client proxies it through its own route, which reads the session cookie server-side.

Salary configuration

Method	Path	Permission
GET	/salary-structures	salaryStructures:read
GET	/salary-structures/:id	salaryStructures:read
POST	/salary-structures	salaryStructures:create
PATCH	/salary-structures/:id	salaryStructures:update
DELETE	/salary-structures/:id	salaryStructures:delete
GET	/salary-rules	salaryRules:read
POST	/salary-rules	salaryRules:create
PATCH	/salary-rules/:id	salaryRules:update
DELETE	/salary-rules/:id	salaryRules:delete

Rules query: `q`, `structureId`, `category`. See [data-model.md](#) for how a rule is evaluated.

Reference data

Method	Path	Permission
GET	/departments	employees:read
POST	/departments	employees:create
PATCH	/departments/:id	employees:update
DELETE	/departments/:id	employees:delete
GET	/job-positions	employees:read
POST	/job-positions	employees:create
PATCH	/job-positions/:id	employees:update
DELETE	/job-positions/:id	employees:delete
GET	/working-schedules	workingSchedules:read
GET	/working-schedules/:id	workingSchedules:read
POST	/working-schedules	workingSchedules:create
PATCH	/working-schedules/:id	workingSchedules:update
DELETE	/working-schedules/:id	workingSchedules:delete
A schedule is written with its <code>lines</code> — the weekly `{ dayOfWeek, startTime, endTime, breakHours }` <code>pattern</code> . <code>hoursPerWeek</code> is always derived from those lines and is never accepted		
from a client.		

Dashboard

Method	Path	Permission
GET	/dashboard	dashboard:read

One aggregate: KPIs, salary by department, a monthly trend, attendance health, time-off totals, and the alert lists the overview links from (missing bank details, no contract, expiring contracts, duplicate payslips, draft pay runs).

Known rough edges

Worth knowing before relying on them:

- `PATCH /attendance/:id` and the time-off update routes accept `employeeId` and silently discard it. The web client omits the control; the DTOs would be more honest without the field.
- `PATCH /attendance/:id` cannot clear `notes` — it uses `??` rather than an `undefined` check.
- `GET /time-off/balances/:employeeId` substitutes an `EMPLOYEE`'s own id for whatever is in the URL rather than answering `403`. It returns the caller's own numbers, so nothing leaks, but a `403` would be the honest answer.
- `DELETE /time-off/types/:id` counts only leave requests before deleting, so a type held solely by allocations fails with a foreign-key error surfaced as a `400`.
- Refusing an allocation does not check consumed leave, so a balance can be driven negative from the API. The web client blocks it; the API is the enforcement point and should too.